

Python Object-Oriented Programming

From Blueprints to Complex Systems

Project

Coding

100-days-of-code

.git

FAQ.md

log.md

r1-log.md

README.md

resources.md

rules.md

atom-packages

browser_persistence

c01

FlashcardsExpress

freecodecamp_tribe

JavaScript-Authentication

.git

models

public

routes

index.js

views

.gitignore

app.js

package.json

README.md

LocalWeatherFCC

node-weather-zipcode

nodeschool

NodeWeather

portfolio

the_authentic

Deloitte

log.md

index.js

```
1 var express = require('express');
2 var router = express.Router();
3 var User = require('../models/user');
4
5 // GET /register
6 router.get('/register', function(req, res, next) {
7   return res.render('register', { title: 'Sign Up' });
8 });
9
10 // POST /register
11 router.post('/register', function(req, res, next) {
12   if (req.body.email &&
13       req.body.name &&
14       req.body.favoriteBook &&
15       req.body.password &&
16       req.body.confirmPassword) {
17     // validate email, name, favoriteBook, password twice
18     if (req.body.password !== req.body.confirmPassword) {
19       var err = new Error('Passwords do not match.');
```

```
20       err.status = 400;
21       return next(err);
22     }
23
24     // create object with form data
25     var userData = {
26       email: req.body.email,
27       name: req.body.name,
28       favoriteBook: req.body.favoriteBook,
29       password: req.body.password
30     };
31
32     // save user to database
33     User.create(userData, function(err, user) {
34       if (err) {
35         return next(err);
36       }
37       return next(user);
38     });
39   } else {
40     return next(new Error('Please fill out all fields.');
```

JavaScript-Authentication-Mongo-Express/routes/index.js 1:1

Module 1: OOP Basics

The Foundation of Classes and Objects

1.1. Introduction to OOP

Object-Oriented Programming (OOP) is a paradigm that organizes software design around **data**, or objects, rather than functions and logic.

- Provides a clear structure to programs
- Makes code easier to maintain, reuse, and debug
- Helps keep your code DRY (Don't Repeat Yourself)
- Allows you to build reusable applications with less code
- **Class:** A blueprint or template for creating objects.
- **Object:** An "instance" of a class. If a class is a blueprint for a house, an object is the actual house built from it.

```
1  # Defining a simple class
2  class MyClass:
3      x = 5
4
5  # Creating an object (Instance)
6  p1 = MyClass()
7  print(p1.x)
8
```

1.2. The `__init__()` Function

To understand classes, you must understand the built-in `__init__()` function. All classes have this function, which is executed when the class is being initiated.

- **The Constructor:** It is used to assign values to object properties or other operations that are necessary to do when the object is being created.

```
1  class Person:
2      def __init__(self, name, age):
3          self.name = name
4          self.age = age
5
6  p1 = Person("John", 36)
7
8  print(p1.name)
9  print(p1.age)
10
```

1.3. The `self` Parameter

The `self` parameter is a reference to the **current instance** of the class.

- **Purpose:** It is used to access variables that belong to the class.
- **Naming:** It does not have to be named `self` ; you can call it whatever you like, but it **must** be the first parameter of any function in the class.

```
1  class Car:
2      def __init__(mysillyobject, brand, model):
3          mysillyobject.brand = brand
4          mysillyobject.model = model
5
6      def drive(abc):
7          print("The " + abc.model + " is driving!")
8
9  c1 = Car("Toyota", "Corolla")
10 c1.drive()
11
```

1.4. Class vs. Instance Properties

Properties (attributes) can be defined at the class level or the instance level.

- **Instance Properties:** Defined inside `__init__` . Unique to each object.
- **Class Properties:** Defined directly in the class. Shared by all instances of that class.

```
1  class Dog:
2      species = "Canis familiaris" # Class Property
3
4      def __init__(self, name, age):
5          self.name = name          # Instance Property
6          self.age = age            # Instance Property
7
8  buddy = Dog("Buddy", 9)
9  miles = Dog("Miles", 4)
10
11  print(buddy.species) # "Canis familiaris"
12  print(miles.species) # "Canis familiaris"
13
```

- **Modify Properties:** You can modify properties on objects like this: `p1.age = 40` .
- **Delete Properties:** You can delete properties using `del` : `del p1.age` .

1.5. Class Methods

Objects can also contain methods. Methods in objects are functions that belong to the object.

```
1  class Playlist:
2      def __init__(self, name):
3          self.name = name
4          self.songs = []
5
6      def add_song(self, song):
7          self.songs.append(song)
8          print(f"Added: {song}")
9
10     def remove_song(self, song):
11         if song in self.songs:
12             self.songs.remove(song)
13             print(f"Removed: {song}")
14
15     def show_songs(self):
16         print(f"Playlist '{self.name}':")
17         for song in self.songs:
18             print(f"- {song}")
19
20 my_playlist = Playlist("Favorites")
21 my_playlist.add_song("Bohemian Rhapsody")
```

1.6. 🧠 Knowledge Check: Basics

```
1  # 1. Constructor Logic
2  class Point:
3      def __init__(self, x, y):
4          self.x = x
5          self.y = y
6
7  p = Point(10, 20)
8  # How do you access the value 10?
9
10 # 2. Self Concept
11 # Can a class method be defined without any parameters?
12 # (A) Yes (B) No
13
14 # 3. Instance vs Class
15 # If you change a Class Property, does it change
16 # for all existing objects of that class?
17
```


Module 2: Advanced OOP

Inheritance, Polymorphism, and Beyond

2.1. Inheritance: Parent and Child

Inheritance allows us to define a class that inherits all the methods and properties from another class.

- **Parent class** (Base class): The class being inherited from.
- **Child class** (Derived class): The class that inherits from another class.

```
1  class Person: # Parent
2      def __init__(self, fname, lname):
3          self.firstname = fname
4          self.lastname = lname
5
6  class Student(Person): # Child
7      pass
8
9  x = Student("Mike", "Olsen")
10 x.printname()
11
```

2.2. The `super()` Function

Python has a `super()` function that will make the child class inherit all the methods and properties from its parent.

```
1 class Student(Person):
2     def __init__(self, fname, lname, year):
3         super().__init__(fname, lname) # Calls parent constructor
4         self.graduationyear = year
5
6     def welcome(self):
7         print("Welcome", self.firstname, "to the class of", self.graduationyear)
8
```

2.3. Polymorphism - 1

The word "polymorphism" means "many forms". In programming, it refers to methods/functions/operators with the same name that can be executed on many objects or classes.

```
1  class Car:
2      def __init__(self, brand, model):
3          self.brand = brand
4          self.model = model
5      def move(self):
6          print("Drive!")
7
8  class Boat:
9      def __init__(self, brand, model):
10         self.brand = brand
11         self.model = model
12     def move(self):
13         print("Sail!")
14
15  class Plane:
16      def __init__(self, brand, model):
17         self.brand = brand
18         self.model = model
19     def move(self):
20         print("Fly!")
```

```
1  class Vehicle:
2      def __init__(self, brand, model):
3          self.brand = brand
4          self.model = model
5      def move(self):
6          print("Move!")
7
8  class Car(Vehicle):
9      pass
10
11  class Boat(Vehicle):
12      def move(self):
13          print("Sail!")
14
15  class Plane(Vehicle):
16      def move(self):
17          print("Fly!")
```

2.3. Polymorphism - 2

```
1  car1 = Car("Ford", "Mustang")      #Create a Car object
2  boat1 = Boat("Ibiza", "Touring 20") #Create a Boat object
3  plane1 = Plane("Boeing", "747")    #Create a Plane object
4
5  for x in (car1, boat1, plane1):
6      print(x.brand)
7      print(x.model)
8      x.move()
```

2.4.1. The Three Levels of Access

Encapsulation is about restricting direct access to data to prevent accidental modification. In Python, we use underscores to indicate the intended "privacy" of a variable.

1. **Public:** No underscore (e.g., `self.name`). Accessible from anywhere.
2. **Protected:** Single underscore (e.g., `self._name`). A convention suggesting it should only be used within the class and its subclasses.
3. **Private:** Double underscore (e.g., `self.__name`). Python's way of making a variable hard to access from outside.

```
1  class Employee:
2      def __init__(self, name, salary):
3          self.name = name          # Public
4          self._project = "Secret"  # Protected
5          self.__salary = salary    # Private
6
7  emp = Employee("Alice", 5000)
8  print(emp.name)                  # Works
9  print(emp._project)              # Works (but discouraged by convention)
10 # print(emp.__salary) # Throws an AttributeError!
11
```

2.4.2. Name Mangling: How "Private" works

When you use a double underscore `__`, Python performs **Name Mangling**. It internally changes the variable name so it is not easily found outside the class.

```
1  class SecretAgent:
2      def __init__(self, name):
3          self.__id = "007"
4
5  agent = SecretAgent("James")
6
7  # This will fail:
8  # print(agent.__id)
9
10 # Python "mangles" the name to: _ClassName__variable
11 # This technically works, but is considered a 'hack' and bad practice:
12 print(agent._SecretAgent__id) # Output: 007
13
```

- **Why use it?** To prevent subclasses from accidentally overriding internal variables that the parent class needs to function correctly.

2.4.3. Getters and Setters (Validation)

The "proper" way to interact with private data is through methods. This allows you to **validate** data before it is saved.

```
1  class Human:
2      def __init__(self, age):
3          self.__age = age
4
5      # Getter: To see the value
6      def get_age(self):
7          return self.__age
8
9      # Setter: To change the value with logic
10     def set_age(self, new_age):
11         if new_age > 0 and new_age < 120:
12             self.__age = new_age
13         else:
14             print("Invalid age provided!")
15
16     person = Human(25)
17     person.set_age(-5)    # Prints: "Invalid age provided!"
18     person.set_age(30)   # Successfully updates
19     print(person.get_age()) # 30
20
```


2.4.4. Practical Example: Secure Bank Account

Encapsulation is critical in financial or security logic. You want the user to change their balance only through approved methods (Deposit/Withdraw), not by typing `account.balance = 99999`.

```
1  class BankAccount:
2      def __init__(self, owner, balance):
3          self.owner = owner
4          self.__balance = balance # Hidden from direct access
5
6      def deposit(self, amount):
7          if amount > 0:
8              self.__balance += amount
9              print(f"Deposited ${amount}. New balance: ${self.__balance}")
10
11     def withdraw(self, amount):
12         if 0 < amount <= self.__balance:
13             self.__balance -= amount
14             print(f"Withdrew ${amount}. Remaining: ${self.__balance}")
15         else:
16             print("Insufficient funds or invalid amount!")
17
18 account = BankAccount("John Doe", 1000)
19 account.deposit(500)
20 account.withdraw(2000) # Prevents negative balance!
21
```

2.5. Inner Classes

An inner class (or nested class) is a class defined inside another class. It is often used to group classes that are only relevant to the outer class.

```
1  class University:
2      def __init__(self, name):
3          self.name = name
4          self.dept = self.Department() # Instantiate inner class
5
6      class Department: # Inner Class
7          def __init__(self):
8              self.name = "Computer Science"
9
10 u = University("MIT")
11 print(u.dept.name)
12
```

End